

Interview Prep — Sweden Java Role (Gap Topics)

Simple, beginner-friendly answers you can build on with your own examples.

1. System Design / Architecture

Q1: Design a URL shortener service — discuss scalability, DB choice, caching.

Simple idea: A URL shortener takes a long URL (e.g. `https://example.com/very/long/path`) and gives back a short one (e.g. `short.ly/abc123`).

How it works: 1. User submits long URL → service generates a short unique code (like `abc123`). 2. Store mapping: `abc123` → `https://example.com/very/long/path` in a database. 3. When someone visits `short.ly/abc123`, service looks up the DB and redirects (HTTP 301/302) to the original URL.

Generating the short code: - Option A: Auto-increment ID converted to Base62 (a-z, A-Z, 0-9) — e.g. ID 125 → "cb". - Option B: Hash the long URL (MD5/SHA) and take first 6-7 chars.

DB choice: - Key-value stores like **DynamoDB** or **Redis** work great because access pattern is simple: key → value lookup. No complex joins needed. - If you need analytics (click counts, etc.), a relational DB (Postgres/MySQL) alongside is fine too.

Scalability: - Reads (redirects) far outnumber writes (new URL creation) → so **cache the popular URLs** (using Redis) to avoid hitting DB every time. - Use a **Load Balancer** in front of multiple service instances (like your student-service is deployed behind ALB). - Shard/partition DB by short-code prefix if data grows huge.

Caching: - Cache `abc123` → long URL in Redis with a TTL. On cache miss, fetch from DB and populate cache.

Relate to your project: Think of it like your `StudentController GET by ID` — instead of DB lookup every time, you'd cache frequently accessed student IDs.

Q2: How would you design a rate limiter for an API gateway?

Simple idea: A rate limiter stops one user/client from calling your API too many times in a short period (e.g., “max 100 requests per minute”).

Common algorithms (explained simply):

1. **Fixed Window Counter:** Count requests in a fixed time window (e.g., 1 minute). If count > limit, reject. Downside: burst at window edges (99 requests at 0:59 + 99 at 1:00 = 198 in 2 seconds).
2. **Sliding Window Log:** Keep timestamps of each request, only count ones within the last X seconds. More accurate but more memory.
3. **Token Bucket:** Imagine a bucket that holds tokens. Each request uses 1 token. Tokens refill at a fixed rate (e.g., 10/sec). If bucket is empty, request is rejected/delayed. This is the most popular approach (used by AWS API Gateway).
4. **Leaky Bucket:** Requests go into a queue (bucket) and are processed at a constant rate, like water leaking out at a fixed speed.

Where to implement (2 levels): - **Gateway level** (AWS API Gateway, Kong, Nginx) — stops abusive traffic **before it even reaches your Spring Boot app**. Best for protecting infrastructure from DDoS/bulk abuse. - **Application level** (inside Spring Boot, using **Resilience4j**) — protects individual endpoints/business logic, gives you fine-grained control per-API or per-client. This is what interviewers usually want you to explain in detail — so here are the exact steps:

Step-by-step: Implementing rate limiting in Spring Boot with Resilience4j

Step 1 — Add the dependency (pom.xml)

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

(spring-boot-starter-aop is required — Resilience4j uses AOP proxies to intercept your annotated methods.)

Step 2 — Configure the rate limiter in application.yml (or .properties)

```
resilience4j:
  ratelimiter:
    instances:
      studentApi:
        limit-for-period: 10           # allow 10 calls
        limit-refresh-period: 1s      # per 1 second window
        timeout-duration: 500ms      # how long a request waits
        for a permit before failing
```

This creates a named rate-limiter instance called `studentApi` that you reference from your code. You can define multiple instances (e.g., one per controller/API) with different limits.

Step 3 — Add the annotation on your Controller/Service method

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @RateLimiter(name = "studentApi", fallbackMethod =
        "rateLimitFallback")
    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudent(@PathVariable
        String id) {
        return ResponseEntity.ok(studentService.getById(id));
    }

    // Step 4 – Fallback method, called automatically when rate
    // limit is exceeded
    public ResponseEntity<Student> rateLimitFallback(String id,
        RequestNotPermitted ex) {
        return ResponseEntity
            .status(HttpStatus.TOO_MANY_REQUESTS)
            .body(null); // or a custom error DTO with a
            friendly message
    }
}
```

- `name = "studentApi"` links this method to the config block you defined in `application.yml`.
- `fallbackMethod` **must have the same return type and parameters as the original method, plus the exception type** (`RequestNotPermitted`) as the last parameter. Spring AOP intercepts the call — if the limit is exceeded, it calls the fallback instead of your actual method.

Step 5 — (Optional) Expose metrics for monitoring

Add `spring-boot-starter-actuator` and enable:

```
management:
  endpoints:
    web:
      exposure:
        include: ratelimiters, ratelimiterevents
```

This lets you see how often the limiter is triggering via `/actuator/ratelimiters` — useful for tuning the limit values in production.

How it works internally: Resilience4j wraps your annotated method in a proxy (via Spring AOP). Every time the method is called, it checks if a “permit” is available (based on `limit-for-period` / `limit-refresh-period`). If yes, the method runs

normally. If no, it waits up to `timeout-duration` for a permit to free up — if still none available, it throws `RequestNotPermitted`, which triggers your fallback method automatically.

Relate to your project: If `StudentController` is public-facing, you'd apply `@RateLimiter` per endpoint (e.g., stricter limit on `POST /students` to prevent spam-creation, looser limit on `GET /students/{id}`). You could also rate-limit **per client/API key** by making the rate-limiter name dynamic or using a custom `KeyResolver` if using Spring Cloud Gateway instead of a single Spring Boot app.

Q3: Explain CAP theorem with a real example — which two would you prioritize for an e-commerce checkout system?

CAP Theorem (simple): In a distributed system, you can only guarantee **2 out of 3**:

- **C — Consistency:** Every read gets the latest write (all nodes show the same data).
- **A — Availability:** System always responds (even if some nodes are down).
- **P — Partition Tolerance:** System keeps working even if network between nodes breaks.

Since network partitions **will** happen in real distributed systems, you must have **P**. So the real choice is between **C** and **A** when a partition occurs.

Real example: - **Banking/Checkout system (payments):** Prioritize **Consistency + Partition Tolerance (CP)**. You'd rather show an error ("try again") than let two people buy the same last item in stock or double-charge a customer. - **Social media likes/comments count:** Prioritize **Availability + Partition Tolerance (AP)**. It's OK if the like-count is slightly outdated for a few seconds — better to always respond than show an error.

For e-commerce checkout specifically: Choose **CP** — during payment/inventory deduction, consistency matters more than availability. You don't want overselling or double-charging just to stay "always available."

Q4: How do you design idempotent APIs? Why is idempotency important in payment systems?

Idempotent (simple definition): Calling the same API multiple times with the same input produces the **same result** — it doesn't create duplicate side effects.

Example: - `GET /students/1` → naturally idempotent (reading doesn't change anything). - `DELETE /students/1` → idempotent (deleting an already-deleted student still results in "student not found", no extra harm). - `POST /payments` → **NOT idempotent by default** — calling it twice could charge the customer twice!

How to make POST idempotent: 1. Client generates a unique **Idempotency-Key** (like a UUID) and sends it in the request header. 2. Server stores this key with the result of the first request. 3. If the same key comes again (e.g., due to network retry), server returns the **stored result** instead of processing the payment again.

Simple flow:

POST /payments
Header: Idempotency-Key: abc-123-uuid

```
Server logic:  
if (key "abc-123-uuid" already processed) {  
    return previous response;  
} else {  
    process payment;  
    store result against key;  
    return response;  
}
```

Why important in payment systems: Network failures cause retries. Without idempotency, a retry could **double-charge** a customer. Idempotency keys ensure retries are safe.

2. Behavioral / Leadership (Swedish interviews — collaborative culture)

Q5: Tell me about a time you disagreed with a technical decision — how did you resolve it collaboratively?

How to answer (structure — STAR method: Situation, Task, Action, Result): - **Situation:** Briefly describe the disagreement (e.g., a colleague wanted to use synchronous REST calls between microservices, you preferred async messaging via SQS/Kafka). - **Task:** Explain what was at stake (latency, reliability). - **Action:** Say you scheduled a discussion, presented pros/cons with data (e.g., latency numbers, failure handling), listened to their reasoning, and together decided on a hybrid approach or a trial/POC. - **Result:** Explain the outcome — improved performance, or you learned something from their perspective too.

Tip for Sweden: Emphasize **collaboration**, not “winning the argument.” Swedish culture values consensus-building over hierarchy — show you value others’ input and aim for the best team outcome, not personal ego.

Q6: How do you mentor junior developers in your team?

Simple answer approach: - Pair programming sessions for tricky bugs. - Code reviews with constructive, specific feedback (not just “this is wrong” but “here’s why and here’s a better way”). - Encourage them to ask “why” and explain reasoning, not just give solutions. - Share learning resources, do small knowledge-sharing sessions. - Give them ownership of small features to build confidence.

Q7: Describe a situation where you had to give/receive critical feedback.

Simple structure: - **Giving feedback:** Focus on the **behavior/code**, not the person. E.g., “This method could cause a NullPointerException in production — let’s add a null check” instead of “You wrote bad code.” - **Receiving feedback:** Show you’re open — “I received feedback that my PR was too large to review effectively. I started breaking down PRs into smaller, focused changes afterward.”

Tip for Sweden: Swedish workplace culture values **direct but respectful** feedback — no dramatic reactions, calm and solution-focused.

Q8: How do you handle work-life balance and team communication in agile sprints? (Sweden emphasizes this)

Simple answer: - I plan my sprint tasks realistically, avoid over-committing during sprint planning. - I communicate blockers early in daily standups rather than working late alone. - I respect team members’ working hours (Sweden values not working overtime/weekends). - I use async communication (Slack/Teams messages) instead of expecting instant replies, respecting people’s focus time.

Tip: Sweden’s work culture strongly values “**Lagom**” (not too much, not too little — balance) and avoiding burnout. Mention that you believe sustainable pace produces better long-term quality than crunch/overtime.

3. Cloud & DevOps (beyond AWS basics)

Q9: Explain Infrastructure as Code — have you used Terraform/CloudFormation?

Simple idea: Instead of manually clicking buttons in AWS Console to create resources (EC2, S3, ALB, etc.), you **write code (config files)** that describes what infrastructure you want. Running this code creates/updates the infrastructure automatically.

Benefits: - Repeatable — same config = same environment every time (dev, staging, prod). - Version-controlled — track changes in Git, just like application code. - No manual errors from clicking wrong options.

Example (Terraform):

```
resource "aws_s3_bucket" "student_bucket" {
  bucket = "student-service-uploads"
}
```

Running `terraform apply` creates this S3 bucket automatically.

CloudFormation is AWS’s own IaC tool (uses YAML/JSON), Terraform is a third-party tool that works across multiple clouds (AWS, Azure, GCP).

If you haven't used it hands-on: Be honest — “I've used AWS Console manually so far, but I understand the concept and have looked into Terraform basics — I'm comfortable learning it quickly since it's declarative and similar to how I already think about infrastructure.”

Q10: How do you monitor and set up alerting for production issues (Prometheus/Grafana)?

Simple idea: - **Prometheus:** Collects metrics (like CPU usage, request count, error rate) from your app at regular intervals (“scraping”). - **Grafana:** A dashboard tool that visualizes Prometheus data as graphs/charts. - **Alerting:** You set thresholds (e.g., “if error rate > 5% for 5 minutes, send alert to Slack/email”).

How your Spring Boot app connects: - Add **Spring Boot Actuator** + **Micrometer** dependency — this exposes a `/actuator/prometheus` endpoint with metrics. - Prometheus scrapes this endpoint periodically. - Grafana reads from Prometheus and shows dashboards.

Relate to your project: You mentioned using **Datadog** for devcap — same concept: Datadog agent collects metrics/logs, and you build dashboards + alerts there instead of Prometheus/Grafana.

Q11: Explain blue-green vs canary deployment with rollback strategy in Kubernetes.

Blue-Green Deployment (simple): - You have 2 environments: **Blue** (current live version) and **Green** (new version). - Deploy new version to Green, test it. - Once confirmed working, switch traffic (e.g., load balancer) from Blue to Green **all at once**. - If something breaks, switch back to Blue instantly (**rollback = just flip traffic back**).

Canary Deployment (simple): - Instead of switching 100% traffic at once, you release the new version to a **small percentage of users first** (e.g., 5%). - Monitor for errors. If all good, gradually increase to 25%, 50%, 100%. - If issues found, roll back that small percentage — much safer, less blast radius.

In Kubernetes: - Blue-Green: Run two Deployments (app-blue, app-green), switch the Service selector to point to the new one. - Canary: Use tools like **Istio** or **Argo Rollouts** to split traffic percentage-wise between old and new pod versions.

Rollback: Kubernetes keeps a revision history (`kubectl rollout undo deployment/my-app`) so you can revert to the previous version quickly if canary/blue-green fails.

Q12: What is a service mesh (Istio)? Why would you use it with microservices?

Simple idea: When you have many microservices talking to each other, you need to handle things like: retries, timeouts, load balancing, security (mTLS), monitoring/tracing — for **every** service-to-service call. Instead of writing this logic into every microservice, a **service mesh** handles it outside your code.

How it works: - A **sidecar proxy** (like Envoy) is deployed alongside every microservice pod. - All network traffic between services goes through these proxies, not directly. - Istio (control plane) configures these proxies centrally — e.g., “route 10% of traffic to v2 of student-service”, “encrypt all traffic between services”, “retry failed calls 3 times”.

Why use it: - Removes repetitive networking code from your business logic (Spring Boot code stays clean). - Centralized traffic control, security, and observability across all microservices. - Easy canary deployments, circuit breaking, without changing app code.

Relate to your project: If devcap has many microservices calling each other, a service mesh would handle retries/timeouts/security between them instead of each service implementing Resilience4j individually.

4. Security

Q13: What is OWASP Top 10 — name a few and how you mitigate them in Spring Boot.

Simple idea: OWASP Top 10 is a list of the **most common web application security risks**, published by a security organization (OWASP), updated periodically.

Common ones + Spring Boot mitigation: 1. **Injection (SQL Injection):** Use **JPA/Prepared Statements** (parameterized queries) instead of building raw SQL strings with user input. 2. **Broken Authentication:** Use Spring Security with strong password hashing (BCrypt), enforce MFA where possible. 3. **Sensitive Data Exposure:** Encrypt sensitive data at rest (DB) and in transit (HTTPS/TLS). Don't log passwords/tokens. 4. **XML External Entities (XXE):** Disable external entity processing in XML parsers. 5. **Broken Access Control:** Use @PreAuthorize/roles in Spring Security to ensure users can only access their own data. 6. **Security Misconfiguration:** Don't expose stack traces/debug info in production; disable unnecessary actuator endpoints. 7. **Cross-Site Scripting (XSS):** Sanitize/escape user input rendered in HTML. 8. **Insecure Deserialization:** Avoid deserializing untrusted data; validate input. 9. **Using Components with Known Vulnerabilities:** Keep dependencies updated (check via mvn dependency-check or Snyk). 10. **Insufficient Logging & Monitoring:** Log security events (failed logins, access denials) for auditing.

Q14: How do you prevent SQL injection and XSS in a Spring Boot app?

SQL Injection prevention: - Always use **JPA repositories** or **PreparedStatement** with `?` placeholders — never concatenate user input directly into SQL strings. - Bad: `"SELECT * FROM students WHERE name = '" + userInput + "'"` ❌ - Good: `@Query("SELECT s FROM Student s WHERE s.name = :name")` with `@Param("name")` ✅ (Spring Data JPA does this automatically)

XSS (Cross-Site Scripting) prevention: - XSS happens when user input (like a comment) contains `<script>` tags that get rendered as executable code in the browser. - Escape/sanitize any user input before rendering it in HTML (Spring's Thymeleaf auto-escapes by default). - Set proper HTTP headers: `Content-Security-Policy, X-XSS-Protection`. - Validate and sanitize inputs on both client and server side.

Q15: Explain OAuth2 flow types — which one for a mobile app vs server-to-server?

Simple idea: OAuth2 is a way to let a user grant limited access to their data to a third-party app **without sharing their password**.

Main flows (grant types):

1. **Authorization Code Flow:** Most common and secure. User logs in via browser, gets redirected back with a temporary “code”, which your backend exchanges for an access token. **Used for web apps and mobile apps** (with PKCE extension for extra security on mobile).
2. **Client Credentials Flow:** No user involved — used for **server-to-server** communication (e.g., one microservice calling another using its own client ID/secret to get a token).
3. **Implicit Flow:** Older, less secure, used to return token directly in URL (mostly deprecated now, replaced by Authorization Code + PKCE).
4. **Resource Owner Password Credentials:** User gives username/password directly to the app (only used in fully trusted first-party apps, generally discouraged now).

Which to use:

- **Mobile app:** Authorization Code Flow **with PKCE** (Proof Key for Code Exchange) — extra protection since mobile apps can't safely store secrets.
- **Server-to-server (microservices):** Client Credentials Flow — no user, just service identity.

5. Testing

Q16: Difference between unit, integration, and contract testing (e.g., Pact)?

Unit Testing: - Tests a **single unit** (one method/class) in isolation, with all dependencies mocked. - Example: Testing `StudentService.calculateGrade()` using Mockito to mock the repository. - Fast, no DB/network involved.

Integration Testing: - Tests how **multiple components work together** — e.g., Controller → Service → Repository → actual/test DB. - Example: `@SpringBootTest` that starts the app context and hits a real (test) database. - Slower but verifies real interactions.

Contract Testing (e.g., Pact): - Used when **two different services/teams** communicate (e.g., student-service calling a school-service API). - Instead of full integration tests (needing both services running), each side defines a **“contract”** — what request/response format is expected. - Consumer (student-service) defines what it expects from the provider (school-service). Pact verifies both sides honor this contract independently — **without needing to run both services together**. - Very useful in microservices to catch breaking API changes early.

Q17: How do you test asynchronous Kafka consumers?

Simple approach: 1. **Unit test the consumer logic** separately — mock the Kafka message and directly call your `@KafkaListener` method with a test message, verify the business logic runs correctly. 2. **Embedded Kafka (integration test):** Use `spring-kafka-test`'s `@EmbeddedKafka` annotation — spins up an in-memory Kafka broker for tests. You publish a test message to a topic and then verify (with `awaitility` or polling) that the consumer processed it correctly (e.g., check DB record was created). 3. **Use `KafkaTemplate` in test** to send messages and a `CountDownLatch` or `Awaitility.await()` to wait for async processing to complete before asserting results.

Example pattern:

```
@EmbeddedKafka(topics = "student-events")
class StudentConsumerTest {
    @Test
    void testConsumeStudentEvent() {
        kafkaTemplate.send("student-events", "test message");
        await().atMost(5, SECONDS).untilAsserted(() ->
            assertThat(studentRepository.count()).isEqualTo(1));
    }
}
```

6. Soft / Culture Fit for Sweden

Q18: How do you approach code reviews — do you focus on strict rules or flexibility?

Simple answer: - I focus on **readability, maintainability, and correctness** more than personal style preferences. - I give suggestions, not commands — e.g., “Consider extracting this into a separate method for readability” instead of “This is wrong.” - I’m flexible on style if the team has agreed conventions (via linter/checkstyle), but strict on things that affect correctness, security, or performance. - I always explain **why**, so it’s a learning opportunity, not just a gate to pass.

Q19: What does “Lagom” (balance) mean to you in software delivery (scope vs quality vs speed)?

Simple answer: “Lagom” means “just the right amount — not too much, not too little.” Applied to software delivery: - Don’t over-engineer a solution for a simple problem (avoid unnecessary complexity/microservices for a small feature). - Don’t rush and skip testing/code quality just to hit a deadline. - Balance **scope, quality, and speed** — deliver a well-tested, simple solution on time rather than a “perfect” over-engineered one late, or a rushed low-quality one fast. - It reflects sustainable, balanced engineering — good enough quality, delivered steadily, without burnout.

Q20: How comfortable are you working in a flat hierarchy with no strict top-down management?

Simple answer: “I’m very comfortable with it. I believe in taking ownership of my work without needing constant direction, raising concerns openly in team discussions, and making decisions collaboratively with peers rather than waiting for a manager to dictate every step. I’ve worked in agile teams where the whole team collectively owns decisions, and I find that this leads to better solutions since everyone contributes ideas equally.”

7. Data & Scalability

Q21: Explain eventual consistency vs strong consistency — example from your project.

Strong Consistency: After a write, **every subsequent read** immediately sees that update, no matter which node/replica you read from. Example: A relational DB transaction — once committed, all reads reflect it.

Eventual Consistency: After a write, it may take some time for all nodes/replicas to sync up. During that window, different reads might return **different (stale) data**, but eventually (after some time) all nodes converge to the same value. Example:

DynamoDB (by default) or S3 — a write might not be visible everywhere instantly.

Real example from your project: - If your StudentService writes a new student to **DynamoDB** and immediately reads it back from a different replica/region, there’s a small chance you get slightly stale data (eventual consistency) unless you explicitly request a **strongly consistent read** (DynamoDB supports this option per-request). - For financial/critical data, you’d want strong consistency; for things like “student view count” (non-critical), eventual consistency is fine and cheaper/faster.

Q22: How do you handle schema migrations in production without downtime (e.g., Liquibase/Flyway)?

Simple idea: Tools like **Liquibase** and **Flyway** let you manage database schema changes (adding columns, tables, etc.) as **version-controlled scripts**, applied automatically when the app starts.

How it works: - You write migration scripts (e.g., `V1__create_student_table.sql`, `V2__add_email_column.sql`). - Flyway/Liquibase tracks which migrations have already run (in a special table) and only applies new ones. - This ensures all environments (dev/staging/prod) have the same schema history.

Avoiding downtime — best practices: 1. **Backward-compatible changes only** in a single deployment — e.g., add a new nullable column instead of renaming/dropping existing ones (old code should still work with the new schema). 2. **Multi-step migrations for breaking changes:** - Step 1: Add new column, deploy app that writes to both old & new column. - Step 2: Backfill data into new column. - Step 3: Deploy app that reads only from new column. - Step 4: Remove old column in a later migration. 3. Avoid long-locking operations (e.g., `ALTER TABLE` on huge tables) during peak hours — run in batches or use online schema change tools.

Q23: Explain database sharding vs partitioning.

Partitioning (simple): Splitting **one large table** into smaller pieces **within the same database**, based on a key (e.g., splitting students table by year — 2023 data in one partition, 2024 in another). Still on the same DB server, just organized more efficiently for faster queries.

Sharding (simple): Splitting data **across multiple separate database servers/instances**, each holding a subset of data. E.g., students with ID 1-1000 on Server A, students with ID 1001-2000 on Server B. Used when a single DB server can't handle the load/data size anymore.

Key difference: Partitioning = same server, organized data. Sharding = multiple servers, distributed data — needed for massive scale (millions/billions of rows) where one server's hardware just isn't enough.

Relate to DynamoDB: DynamoDB automatically shards your data across partitions based on your **partition key** — this is why choosing a good partition key (e.g., `studentId` with high cardinality) matters for even data distribution and performance.

8. Java Ecosystem Modern Topics

Q24: What's new in Java 17/21 (records, sealed classes, virtual threads)?

Records (Java 16+): A quick way to create immutable data classes without writing boilerplate (constructor, getters, equals, hashCode, toString).

```
record Student(String name, int age) {}  
// Automatically gives: getters (name(), age()), equals(),  
hashCode(), toString()
```

Sealed Classes (Java 17): Restrict which classes can extend/implement a class or interface — gives more control over your class hierarchy.

```
public sealed interface Shape permits Circle, Square {}  
public final class Circle implements Shape {}  
public final class Square implements Shape {}  
// Only Circle and Square are allowed to implement Shape – nothing  
else.
```

Useful with pattern matching (switch expressions) to ensure all cases are handled.

Virtual Threads (Java 21 - Project Loom): Lightweight threads managed by the JVM (not OS threads). You can create **millions** of them cheaply, great for high-concurrency apps (e.g., handling many simultaneous API requests) without the overhead of traditional OS threads.

```
Thread.startVirtualThread(() -> {  
    // do some blocking I/O work, cheaply  
});
```

This makes traditional “one thread per request” thread pools scale much better since virtual threads don’t consume heavy OS resources.

Other useful additions: Pattern matching for switch, text blocks (""multi-line strings""), improved NullPointerException messages (helpful NPE), enhanced garbage collectors (ZGC improvements).

Q25: Explain reactive programming — Mono/Flux basics, when to use WebFlux over MVC.

Simple idea: Traditional Spring MVC is **blocking** — each request occupies a thread until it fully completes (including waiting for DB/network calls). Reactive programming (Spring WebFlux) is **non-blocking** — a thread can handle other work while waiting for a slow operation (like a DB call) to finish, then resume when data is ready.

Mono and Flux (Project Reactor — used by WebFlux): - **Mono:** Represents **0 or 1** result asynchronously. E.g., `Mono<Student>` — fetching a single student. - **Flux:** Represents **0 to many** results asynchronously (a stream of items). E.g., `Flux<Student>` — fetching a list of students, emitted one at a time.

Simple example:

```
public Mono<Student> getStudent(String id) {  
    return studentRepository.findById(id); // returns  
        Mono<Student>  
}  
  
public Flux<Student> getAllStudents() {  
    return studentRepository.findAll(); // returns Flux<Student>  
}
```

You don't `.get()` the value immediately — you chain operations (`.map()`, `.filter()`, `.flatMap()`) and the actual execution happens later when someone “subscribes” (e.g., when Spring Boot returns the response to the client).

When to use WebFlux over traditional MVC: - **High-concurrency systems** where you have many simultaneous requests, especially with slow I/O (calling external APIs, slow DB), and you want to avoid running out of threads. - **Streaming data** — e.g., real-time notifications, chat, live updates. - **NOT needed** for simple CRUD apps with low-medium traffic — MVC (blocking, easier to write/debug) is usually simpler and sufficient (like your `StudentController` likely doesn't need WebFlux unless you expect huge concurrent load).

Trade-off: Reactive code is **harder to write, read, and debug** (different mindset, harder stack traces) — only use it when you actually need the scalability benefit.

Good luck with your interviews! Review these with your own project examples (student-service, devcap) to make answers more personal and credible.